

Smart contract audit rain.math.saturating

Content

Project description	3
Executive summary	3
Reviews	3
Technical analysis and findings	4
Security findings	5
**** Critical	5
*** High	5
** Medium	5
* Low	5
Informational	5
I01 - Ambiguous NatSpec Documentation for Saturating Operations	5
Approach and methodology	6



Project description

This repository contains a Solidity library for saturating arithmetic. It provides safety against overflow and underflow by clamping values at the numeric bounds (0 or `type(uint256).max`) instead of reverting or wrapping around. It is designed to be safer for certain asset transfer logic where small calculation errors shouldn't lock funds.

Executive summary

Type	Library
Languages	Solidity
Methods	Architecture Review, Manual Review, Unit Testing, Functional Testing, Automated Review
Documentation	README.md
Repositories	https://github.com/rainlanguage/rain.math.saturating

Reviews

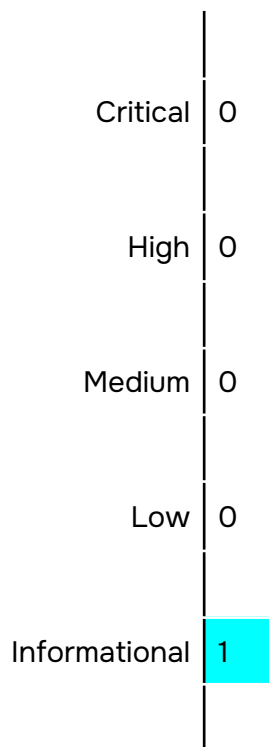
Date	Repository	Commit
13/01/26	rain.math.saturating	22e58d70a223b18d83dfc5529e3ac87aef3ff478
26/01/26	rain.math.saturating	ce2d05c8be329629f8f8c1c3f052480e836361db

Scope

Contracts
src/lib/LibSaturatingMath.sol



Technical analysis and findings



Security findings

**** Critical

No critical severity issue found.

*** High

No high severity issue found.

** Medium

No Medium severity issue found.

* Low

No Low severity issue found.

Informational

I01 - Ambiguous NatSpec Documentation for Saturating Operations

The NatSpec documentation of LibSaturatingMath provides only minimal descriptions of the return values (e.g. "*Minimum of $a + b$ and $\max \text{uint256}$* "), which may be insufficient or ambiguous for third-party developers relying on the library without reviewing the implementation in detail.

Path: src/lib/LibSaturatingMath.sol

Recommendation: Improve the NatSpec documentation by explicitly and exhaustively describing the behavior of each function, including:

- the fact that operations never revert,
- the exact saturation behavior on overflow and underflow.

Found in: 22e58d70

Status: Fixed at e58ce8be



Approach and methodology

To establish a uniform evaluation, we define the following terminology in accordance with the OWASP Risk Rating

Methodology:

	Likelihood Indicates the probability of a specific vulnerability being discovered and exploited in real-world scenarios
	Impact Measures the technical loss and business repercussions resulting from a successful attack
	Severity Reflects the comprehensive magnitude of the risk, combining both the probability of occurrence (likelihood) and the extent of potential consequences (impact)

Likelihood and impact are divided into three levels: High H, Medium M, and Low L. The severity of a risk is a blend of these two factors, leading to its classification into one of four tiers: Critical, High, Medium, or Low.

When we identify an issue, our approach may include deploying contracts on our private testnet for validation through testing. Where necessary, we might also create a Proof of Concept PoC to demonstrate potential exploitability. In particular, we perform the audit according to the following procedure:

	Advanced DeFi Scrutiny We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs
	Semantic Consistency Checks We then manually check the logic of implemented smart contracts and compare with the description in the white paper.
	Security Analysis The process begins with a comprehensive examination of the system to gain a deep understanding of its internal mechanisms, identifying any irregularities and potential weak spots.